Práctica 1. Organización de ficheros

Autor: Lucas Hidalgo Herrera

Grado: Doble Grado en Ingeniería Informática y Matemáticas

Asignatura: Administración de Bases de Datos

Profesor: Ignacio José Blanco Medina

Fecha: 15 de marzo de 2026

Índice General

1 Introducción **1**

2 TDA fileRecord **1**

2.1 Estructura en memoria 1

2.2 Operaciones públicas 1

1 Introducción

A lo largo de este documento se va a realizar la práctica 1 de la asignatura de *Administración de Bases de Datos*. Dicha práctica consiste en documentar, diseñar y documentar algunos tipos de datos abstractos para el trabajo con ficheros de carácter secuencial físico. Estos tipos de datos abstractos están basados en la estructura en memoria de los ficheros del sistema operativo; que como vimos en clase, un fichero no es mas que una secuencia de bloques junto a una cabecera, donde cada bloque es una secuencia de registros precedidos globalmente por una cabecera. De cada uno de ellos, trataremos la *estructura en memoria*, las *operaciones públicas* y las *operaciones privadas* [1]. Por tanto, debemos especificar y diseñar los tipos de datos abstractos **TDA fileASF** y **TDA fileRecord**.

2 TDA fileRecord

Comenzamos por el más interno, lo que ayudará a aprender de forma constructiva el objetivo de la práctica.

2.1 Estructura en memoria

Siguiendo el guión de la práctica, un registro es una secuencia de bytes que almacena dos aspectos, el primero de ellos es la estructura propia del registro [2] y el segundo son los datos siguiendo la estructura que se ha impuesto en el primer dato.

```
// Representación en memoria del registro
private:
estructura fileRecord {
    const entero headerSize;
    const entero numAttr;
    const secuencia<entero> tipoCampos;
    const secuencia<entero> camposSize;
    secuencia<bytes> content;
}
```

Listing 1: Especificación de la estructura del registro en memoria

Como vemos, aunque en el enunciado del problema se determina que los atributos *headerSize*, *numAttr*, *tipoCampos* y *camposSize* están unificados para todos los registros, considero que debemos almacenarlos en cada registro puesto que hacen que este tipo de dato abstracto sea autodescriptivo para posibles tareas necesarias y encapsulado en la memoria *RAM*.

2.2 Operaciones públicas

Puesto que uno de los requisitos que se nos impone en la práctica es que el contenido de los registros pueda ser obtenido por un objeto de tipo *TDA fileASF*, debemos

[1] Todo ello en pseudocódigo

[2] Con esto estamos dando la mayor libertad posible sobre la estructura de los registros, es decir, estamos haciendo el tipo de dato abstracto más general posible

imponer algunas de las funciones de tipo *get* y tipo *set*.

```
// Getter y Setters
public:
/**
* Getter para obtener el tamaño de la cabecera del registro
*
* @ret: copia del tamaño de la cabecera
**/
entero getHeaderSize() const;

/**
* Getter para obtener el numero de atributos
*
* @ret: copia del numero de atributos
**/
entero getNumAttr() const;

/**
* Getter para obtener el tipo de todos los campos
*
* @ret: copia de la secuencia del tipo de los campos
**/
secuencia<entero> getTipoCampos() const;

/**
* Getter para obtener el tamaño de todos los campos
*
* @ret: copia de la secuencia del tamaño de los campos
**/
secuencia<entero> getCamposSize() const;

/**
* Getter para obtener el contenido de todo el registro
*
* @ret: copia del contenido del registro
**/
secuencia<bytes> getContent() const;

/**
* Setter para cambiar el contenido de todo el registro
*
* @param newContent: secuencia de bytes que alberga el nuevo
    contenido del registro
*
* @pre: content.size()==newContent.size
* @post: No se cambia el tamaño del registro, pero sí su contenido
**/
void setContent(secuencia<bytes> newContent);
```

Listing 2: Constructor y destructor del registro

Con respecto al único *setter* que he creado, he supuesto que los datos del nuevo contenido tienen el mismo tipo; aún así, si no lo fuera, considero que la base ded atos proporcionaría el error. Entonces, en caso de que fuera necesario, se podría imponer la condición *newContent[i].type==tipoCampos[i]*, pero no soy muy partidario de

este tipo de prácticas. Para finalizar este aspecto, siguiendo la estructura que se nos propone en el ejercicio, no debemos realizar ningún otro *Setter*, ya que tratamos con bloques de estructura **homogénea** y con registros de longitud fija, es decir, cambiar *camposSize*, *headerSize*, *tipoCampos* o *numAttr* está en contra de esta estructura.

Como aclaración, el motivo por el cual todos los *getter* devuelven una copia es para evitar posibles modificaciones por referencia dependiendo del lenguaje de programación que se use; verificando así que no se pueden modificar datos que no se deban modificar.

Por último, debemos crear un constructor y un destructor de este tipo de dato. Cabe recalcar que, pensando en que estamos creando una estructura de datos versátil, no debe influir que su uso está destinado a ser contenido por el tipo *TDA fileASF*.

```
// Representación en memoria del registro
public:
/**
* Crea una instancia del tipo fileRecord con los datos pasados como
    argumentos
* @param hSize : tamaño de la cabecera del registro
* @param nAttr : numero de atributos del registro
* @param tCampos: secuencia de bytes con el tipo de los campos
* @param cSize: tamaño de cada uno de los campos
* @param cont: contenido de cada uno de los campos del registro
*
* @pre: hSize>0, nAttr>0
* @post: Se crea una instancia de tipo fileRecord
**/
fileRecord (entero hSize, entero nAttr, secuencia<entero> tCampos,
    secuencia<entero> cSize, secuencia<bytes> content);

/**
* Destruye una instancia de fileRecord
*
* @post: Se crea una instancia de tipo fileRecord
**/
~fileRecord()
```

Listing 3: Constructor y destructor del registro

2.3 Operaciones Privadas

Como cabia de esperar, considero que este tipo de dato no admite operaciones privadas debido a que no dispone de una funcionalidad activa. Es decir, es claro que la principal funcionalidad de este tipo de dato es portar información de memoria a disco y de disco a memoria.

Por tanto, desde mi punto de vista, las operaciones privadas exigidas para el manejo del disco se delegan al tipo de dato abstacto *TDA fileASF*.

No obstante, podríamos pensar en una operación privada que realice una de las comprobaciones más importantes, la verificación de tamaños en la creación de una instancia de este tipo de dato; tendría la siguiente especificación:

```
// Operaciones privadas
private:
/**
* Comprueba las restricciones de que los tamaños y cantidades sean
    positivos así como que el tamaño del contenido sea igual a la
    suma de los tamaños especificados en camposSize al estar en un
    ASF. Además verifica que haya tantos tipos de campos como numero
     de atributos especificados. En definitiva, realiza las
    comprobaciones necesarias para no crear un objeto incorrecto.
*
* @param hSize : tamaño de la cabecera del registro
* @param nAttr : numero de atributos del registro
* @param tCampos : tipo de los campos del registro
* @param cSize: tamaño de cada uno de los campos
* @param cont: contenido de cada uno de los campos del registro
*
* @ret: true -> tamaños todos positivos y dentro de las
    restricciones
**/
boolean checkSizes (entero hSize, entero nAttr, secuencia<entero>
    tCampos, secuencia<entero> cSize, secuencia<bytes> content);
```

Listing 4: Operaciones privadas del registro